

## Set up a python project

1. [Install uv](#)
2. Create a [project](#)

- **New project:** `uv init my-project` (creates a `my-project` folder).
- **Existing project:** Run `uv init` inside the project folder.
- **Library project:** Use `uv init --lib` to set up a library structure (includes a `src/` directory).

### Bash

➤ `uv init my_project`

➤ `cd my_project`

This command automatically generates the following files and structure:

- `pyproject.toml` : The standard configuration file for project metadata and dependencies.
- `.gitignore` : Pre-populated with sensible defaults.
- `.python-version` : Specifies the default Python version for the project.
- `README.md` : An empty starting point for project documentation.
- `main.py` : A simple "Hello, world!" script as an entry point.
- A local Git repository is also initialized by default.

uv manages python versions automatically.

- **Install versions:** `uv python install 3.13` downloads a specific version if missing

3. Add dependency (e.g. pytest)

- `uv add requests`
- `uv add pytest --dev`

4. Install all dependencies

The first time you run a command like `uv run` or `uv sync`, it automatically creates a virtual environment in a `.venv` folder and installs all dependencies.

You can also manually [synchronize](#) the env.

- `uv sync`

5. Run your project

uv ensures the environment is in sync before execution.

- `uv run main.py`

6. [Build and publish](#) (**optional step**, if you're building a package for distribution via PyPI)

## Integrations

1. [Using uv in Docker](#) – find a [dockerfile example](#)
2. [Using uv in GitHub Actions](#)